

High Performance Scientific Computing in Aerospace - Module 1

Daniele Bergamaschi, 10713052

Inserire data di consegna

1 Introduction

In the context of *Advanced Air Mobility*, electronic components play a crucial role in ensuring both the safety and the proper operation of the aircraft. One of the main challenges to be addressed is temperature management. Electronic devices that experience excessive heating can lead, over time, to reduced performance, safety risks, and higher maintenance costs. Therefore, it becomes essential to design components whose cooling systems are as efficient as possible. In particular, the geometry of these systems should be optimized to exploit both external and internal airflow for improved heat dissipation.

The objective of this project is to develop the numerical core solver that serves as the foundation for a topology optimization framework applied to power electronics in Advanced Air Mobility. The software was developed entirely in C++, using the MPI paradigm for parallelization. (Aggiungere altri dettagli magari : librerie principali usate)

2 Mathematical problem

The mathematical problem we aim to solve concerns the incompressible Navier–Stokes–Brinkman equations within a cubic domain. In our implementation, we will consider a simplified linear version, valid for low-velocity flows where viscous effects dominate and the convective term $(\mathbf{u} \cdot \nabla)\mathbf{u}$ can be neglected. The time-dependent system can therefore be written as follows:

$$\begin{cases} \frac{\partial \mathbf{u}}{\partial t} - \nu \nabla^2 \mathbf{u} + \frac{\nu}{\kappa} \mathbf{u} + \frac{1}{\rho} \nabla p = \mathbf{f}, \\ \nabla \cdot \mathbf{u} = 0 \end{cases} \quad (1)$$

The first equation corresponds to the conservation of momentum and the terms are as follows :

- Velocity vector $\mathbf{u} = (u, v, w)$, representing the fluid velocity in three directions.
- Pressure p , representing the internal forces due to compression.
- Fluid density ρ , assumed to be constant.
- Kinematic viscosity ν , where $\nu = \frac{\mu}{\rho}$ and μ is the dynamic viscosity.
- The permeability coefficient κ , which is related to the porosity of the medium.

The second equation, $\nabla \cdot \mathbf{u} = 0$, represents the incompressibility condition, which ensures the conservation of mass. It should be noted that the permeability coefficient κ is very important for topology optimization. By changing κ , different regions can be modeled, allowing for the design of optimized structures.

Finally, the available data for our problem include:

- Initial condition $\mathbf{u}(\mathbf{0})$ (da migliorare).
- Boundary condition.
- External body forces $\mathbf{f}$.

3 Numerical implementation

To develop a program capable of solving the problem, Eq. (1) needs to be reformulated in algebraic form. Accordingly, the equation was discretized in time using the Crank-Nicholson scheme and in space using the finite difference method. This choice was motivated by the lower computational cost and the simplicity of implementation offered by finite difference methods compared to other techniques, at the expense of solution accuracy. Finally, a fractional step method was employed to solve the equations, achieving second-order accuracy for the overall procedure.

3.1 Introduction to the Finite Difference Method

The *finite difference method* is a numerical technique used to approximate the derivatives of a function by means of discrete function values. Starting from the definition of the derivative, replacing the limit with a finite increment Δx leads to the *forward difference* approximation, which is first-order accurate, since the truncation error is proportional to Δx .

Because achieving an acceptable accuracy with a first-order method would require a very small step size, it is preferable to increase the order of accuracy by using additional grid points. In particular, by considering three points symmetrically located around x_0 and exploiting Taylor series expansions, one obtains the *central difference formula* for the first derivative, which is second-order accurate:

$$f'(x_0) \approx \frac{f(x_0 + \Delta x) - f(x_0 - \Delta x)}{2 \Delta x}.$$

The same approach can be applied to derive a three-point, second-order accurate approximation of the second derivative:

$$\frac{d^2 f}{dx^2}(x_0) \approx \frac{f(x_0 - \Delta x) - 2f(x_0) + f(x_0 + \Delta x)}{(\Delta x)^2}.$$

In conclusion, the use of central difference schemes significantly improves the accuracy without excessively increasing the number of grid points, making the finite difference method particularly effective for the discretization of differential problems.

3.2 The spatial discretization of the system

To understand the final formulation of the spatially discretized problem, we begin by considering the discretization of the pressure gradient ∇p .

Let us consider a three-dimensional cubic domain discretized into a grid, where the center of each cell is identified by the index triplet (i, j, k) . Using the finite difference method introduced in the previous section, the discrete approximation of the pressure gradient at the cell (i, j, k) becomes

$$\nabla p|_{i,j,k} \approx \left[\frac{p_{i+1,j,k} - p_{i-1,j,k}}{2 \Delta x}, \frac{p_{i,j+1,k} - p_{i,j-1,k}}{2 \Delta y}, \frac{p_{i,j,k+1} - p_{i,j,k-1}}{2 \Delta z} \right].$$

where the values $p_{i,j,k}$ represent the unknown pressure values to be determined at each grid cell.

Discretizing the pressure in this manner can lead to numerical stability issues. To illustrate, consider only the points along the x-direction. The pressure at point i is approximated using the values at points $i - 1$ and $i + 1$. As a result, if i is even, only the odd-indexed points are used in the approximation, and vice versa. This creates a decoupling between even and odd grid points.

While this decoupling is not inherently problematic, the compatibility conditions for pressure indicate that if p is a solution, then $p + \text{constant}$ is also a solution. In the presence of decoupling, it becomes possible to introduce two different constants—one for even points and one for odd points—leading to a final pressure field that may oscillate between the two sets. This phenomenon, known as *checkerboarding*, can render the numerical solution unstable.

To address this issue, the pressure values are stored at the standard grid points, while the velocity components are located at the cell faces, shifted by half a grid step, as illustrated in Fig. 1. This type of arrangement is known as a staggered grid.

Using a staggered grid the discretization becomes as follow :

- **u -momentum equation at an x-face, index $(i + 1/2, j, k)$:**

$$\frac{u_{i+\frac{1}{2},j,k}^{n+1} - u_{i+\frac{1}{2},j,k}^n}{\Delta t} - \mu \Delta_h u_{i+\frac{1}{2},j,k} + \frac{\mu}{\kappa} u_{i+\frac{1}{2},j,k} + \frac{p_{i+1,j,k} - p_{i,j,k}}{\Delta x} = f_{i+\frac{1}{2},j,k}^x, \quad (2)$$

where the discrete Laplacian Δ_h is defined as:

$$\Delta_h u_{i+\frac{1}{2},j,k} = \frac{u_{i+\frac{3}{2},j,k} - 2u_{i+\frac{1}{2},j,k} + u_{i-\frac{1}{2},j,k}}{(\Delta x)^2} \quad (3)$$

$$+ \frac{u_{i+\frac{1}{2},j+1,k} - 2u_{i+\frac{1}{2},j,k} + u_{i+\frac{1}{2},j-1,k}}{(\Delta y)^2} \quad (4)$$

$$+ \frac{u_{i+\frac{1}{2},j,k+1} - 2u_{i+\frac{1}{2},j,k} + u_{i+\frac{1}{2},j,k-1}}{(\Delta z)^2}. \quad (5)$$

- **Divergence constraint at the pressure point (i,j,k) (cell center):**

$$\frac{u_{i+\frac{1}{2},j,k} - u_{i-\frac{1}{2},j,k}}{\Delta x} + \frac{v_{i,j+\frac{1}{2},k} - v_{i,j-\frac{1}{2},k}}{\Delta y} + \frac{w_{i,j,k+\frac{1}{2}} - w_{i,j,k-\frac{1}{2}}}{\Delta z} = 0.$$

Figure 1: Example of a 2-dimensional staggered grid (left). The black dots represent pressure points, blue arrows indicate the u component of velocity, and red arrows indicate the v component of velocity. The right diagram zooms into a single grid point (i,j).

3.3 Time Discretization

For the time discretization, the Crank–Nicolson method has been adopted. This scheme belongs to the family of linear multistep methods, whose general formulation is

$$\sum_{j=0}^r \alpha_j U_{n+j} = \Delta t \sum_{j=0}^r \beta_j f(U_{n+j}, t_{n+j}),$$

where the coefficients α_j define the linear combination of the solution values at different time levels, while the coefficients β_j weight the evaluations of the function f . A multistep method is explicit if $\beta_r = 0$, and implicit otherwise.

Crank–Nicolson can be interpreted as a particular case of the one-step ($r = 1$) Adams–Moulton family. By choosing

$$\alpha_1 = 1, \quad \alpha_0 = -1,$$

the scheme reduces to

$$U_{n+1} = U_n + \Delta t \sum_{j=0}^1 \beta_j f(U_{n+j}, t_{n+j}).$$

Imposing that the evaluation of the function is the average between the current and the next time level leads to

$$\beta_0 = \frac{1}{2}, \quad \beta_1 = \frac{1}{2},$$

which yields the Crank–Nicolson method:

$$U_{n+1} = U_n + \frac{\Delta t}{2} [f(U_n, t_n) + f(U_{n+1}, t_{n+1})].$$

The choice of this method is motivated by its favourable numerical properties: Crank–Nicolson is second–order accurate in time and it is A-stable. The Navier–Stokes–Brinkman equations define a problem that is typically stiff, due to the presence of the porous-medium resistance term, which dominates the dynamics and introduces slow temporal scales in the system. For this reason, using a time-integration scheme that is not A-stable would require selecting an extremely small time step to maintain numerical stability, leading to a substantial increase in computational cost. A-stability therefore becomes a necessary requirement to effectively handle the stiffness of the system, allowing the use of larger time steps without compromising the stability of the scheme. Furthermore, being a one-step method, the update from U_n to U_{n+1} depends only on the solution at the previous time level, which reduces the memory requirements and simplifies the implementation.

3.4 Direction-splitting Method for NSB

The discretization of the Navier–Stokes–Brinkman equations in a three dimensional domain leads to a linear system of extremely large size. For a grid with approximately $N \approx 10^8$ discretization points, the resulting matrix becomes practically impossible to handle using direct solvers, while standard iterative methods would still require computational and memory resources that are not feasible.

To make the problem tractable, it is therefore necessary to introduce a decoupling strategy that decomposes the original system into smaller subsystems. Two main approaches exist: exact decoupling, which preserves the structure of the equations but entails very high computational costs, and approximate decoupling, based on operator-splitting techniques.

In the present problem, exact decoupling would be computationally too expensive and would make the efficient use of preconditioners difficult. For this reason, an uncoupled approximation method belonging to the family of fractional-step methods has been adopted, specifically a direction-splitting method, which reduces the three-dimensional problem to a sequence of one-dimensional problems, making the algorithm significantly more efficient in terms of memory usage and computational time.

3.4.1 Method description

The method is developed starting from a perturbed formulation of the Navier-Stokes-Brinkman equations. Following the approach introduced by Guermond, Mineev, and Shen, a perturbation parameter $\varepsilon = \Delta t$ is introduced, leading to the following modified system:

$$\begin{cases} \frac{\partial u^\varepsilon}{\partial t} - \nu \nabla^2 u^\varepsilon + \frac{\nu}{k} u^\varepsilon + \nabla p^\varepsilon = f, & u^\varepsilon|_{\partial\Omega} = 0, \\ -\varepsilon \nabla^2 \phi^\varepsilon + \nabla \cdot u^\varepsilon = 0, & \frac{\partial \phi^\varepsilon}{\partial n} \Big|_{\partial\Omega} = 0, \\ \varepsilon \frac{\partial p^\varepsilon}{\partial t} = \phi^\varepsilon - \chi \nu \nabla \cdot u^\varepsilon. \end{cases}$$

As $\varepsilon \rightarrow 0$, one can show that $u^\varepsilon \rightarrow u$ and $p^\varepsilon \rightarrow p$, i.e., the solutions of the perturbed system converge to the velocity and pressure fields of the original Navier-Stokes-Brinkman problem. Moreover, the introduction of the auxiliary variable ϕ^ε makes it possible to separate the velocity and pressure updates, effectively decoupling the problem. The approximation error introduced by this perturbation is of order $\mathcal{O}(\varepsilon^2)$.

A key theoretical result is that the Laplace operator in the perturbed formulation can be replaced by a more general operator A , provided that A satisfies the following two properties:

1. The bilinear form

$$a(p, q) = \int_{\Omega} q A p \, dx$$

is symmetric.

2. A coercivity estimate holds:

$$\|q\|_2^2 \leq a(q, q), \quad \forall q \in D(A).$$

These conditions ensure the stability and convergence of the scheme. With the operator A replacing the Laplacian, the perturbed system becomes:

$$\begin{cases} \frac{\partial u^\varepsilon}{\partial t} - \nu \nabla^2 u^\varepsilon + \frac{\nu}{k} u^\varepsilon + \nabla p^\varepsilon = f, & u^\varepsilon|_{\partial\Omega} = 0, \\ -\varepsilon A \phi^\varepsilon + \nabla \cdot u^\varepsilon = 0, & \frac{\partial \phi^\varepsilon}{\partial n} \Big|_{\partial\Omega} = 0, \\ \varepsilon \frac{\partial p^\varepsilon}{\partial t} = \phi^\varepsilon - \chi \nu \nabla \cdot u^\varepsilon. \end{cases} \quad (6)$$

The operator A adopted in this work, and which satisfies the requirements stated above, is defined as

$$A = -(1 - \partial_{xx})(1 - \partial_{yy})(1 - \partial_{zz}).$$

This operator is particularly appealing because solving the linear system $Ap = f$ resulting from the discretisation reduces to solving a cascade of three one-dimensional problems, requiring only $\mathcal{O}(N^3)$ operations:

$$\psi - \partial_{xx}\psi = -\frac{1}{\Delta t} \nabla \cdot u^{n+1}, \quad \partial_x \psi|_{x=0,1} = 0$$

$$\varphi - \partial_{yy}\varphi = \psi, \quad \partial_y \varphi|_{y=0,1} = 0$$

$$\phi^{n+1/2} - \partial_{zz}\phi^{n+1/2} = \varphi, \quad \partial_z \phi|_{z=0,1} = 0.$$

These three equations are fully decoupled, each associated with a single spatial direction, and can be solved sequentially in a directional cascade: once the first equation is solved, its solution becomes the forcing term for the second one, and analogously for the third.

Moreover, the discretisation of the operators of the form $(I - \partial_{xx})$, using the numerical techniques discussed in the previous sections, leads to linear systems of the form $Ax = b$ in which the matrix A has a block structure. This greatly reduces the computational cost, making the overall scheme extremely efficient. These equations allow updating the pressure.

To advance the velocity, and to achieve a second-order accurate time scheme, the Crank–Nicolson method is applied for the temporal discretisation, as described in the previous section. This leads to the following intermediate velocity equation:

$$\frac{u^* - u^n}{\Delta t} + \left(\frac{\nu}{2\kappa} - \frac{\nu}{2}\nabla^2\right)(u^* + u^n) = f^{n+1/2} - \nabla p_\star^{n+1/2}$$

where the intermediate pressure is defined as

$$p_\star^{k+1/2} = p^{k-1/2} + \phi^{k-1/2}.$$

By adding and subtracting u^n in the second term and rearranging terms, the equation can be rewritten in the compact form as :

$$(1 + \Delta t \frac{\nu}{2\kappa} - \Delta t \frac{\nu}{2}\nabla^2)(u^* - u^n) = \Delta t g^{n+1/2}$$

By renaming the term $(1 + \Delta t \frac{\nu}{2\kappa} - \Delta t \frac{\nu}{2} \nabla^2)$ as $(1 - \gamma \nabla^2)$, the previous equation can be rewritten as:

$$(1 - \gamma \nabla^2)(u^* - u^n) = \Delta t \beta g^{n+1/2},$$

where

$$\gamma = \frac{\Delta t \nu}{2\beta}, \quad \beta = 1 + \frac{\Delta t \nu}{2} \kappa.$$

Finally, by approximating the operator as

$$(1 - \gamma \nabla^2) \approx (1 - \gamma \partial_{xx})(1 - \gamma \partial_{yy})(1 - \gamma \partial_{zz}),$$

the velocity can be computed in $O(N^3)$ operations by solving the following 1D equations in cascade:

$$\xi^{n+1} - u^n = \Delta t \beta \hat{g}^{n+1/2}$$

$$\eta^{n+1} - \xi^{n+1} - \gamma \partial_{xx}(\eta^{n+1} - \eta^n) = 0,$$

$$\zeta^{n+1} - \eta^{n+1} - \gamma \partial_{yy}(\zeta^{n+1} - \zeta^n) = 0,$$

$$u^{n+1} - \zeta^{n+1} - \gamma \partial_{zz}(u^{n+1} - u^n) = 0,$$

where

$$\hat{g}^{n+1/2} = f^{n+1/2} - \nabla p_\star^{n+1/2} - \frac{\nu}{2} \kappa u^n + \frac{\nu}{2} (\partial_{xx} \eta^n + \partial_{yy} \zeta^n + \partial_{zz} u^n).$$

Although the method has been described here for the u -component of the velocity field, it is applied in the same manner to the remaining components, v and w , independently.

4 Boundary Conditions in a Staggered Grid

In a staggered grid framework utilizing direction splitting, the momentum equations are solved by advancing intermediate velocity vectors, denoted as η , ζ , and u . Because the grid is staggered, the physical boundaries do not treat all variables symmetrically: the left boundary (near the origin) coincides with pressure nodes, while the right boundary coincides with normal velocity nodes.

4.1 Velocity Boundary Conditions

The method of imposition depends on whether the velocity component is parallel or tangential to the direction of the current sweep.

4.1.1 Parallel Component (e.g., u in the X-sweep)

When solving for the velocity component aligned with the sweep direction:

- **Left Boundary (Origin):** The first computational node for u is located at $i = 1/2$, which is staggered by $\Delta x/2$ from the physical wall. To maintain second-order accuracy, the incompressibility constraint ($\nabla \cdot \mathbf{u} = 0$) is used to find the derivative at the wall:

$$\left. \frac{\partial u}{\partial x} \right|_{0,j,k} = -\left. \frac{\partial v}{\partial y} \right|_{0,j,k} - \left. \frac{\partial w}{\partial z} \right|_{0,j,k} \approx -\frac{v_{0,j+1/2,k} - v_{0,j-1/2,k}}{\Delta y} - \frac{w_{0,j,k+1/2} - w_{0,j,k-1/2}}{\Delta z} \quad (7)$$

The value at the staggered node is then computed using a second-order shift:

$$u_{1/2,j,k} = u_{0,j,k} + \left. \frac{\partial u}{\partial x} \right|_{0,j,k} \frac{\Delta x}{2} \quad (8)$$

- **Right Boundary:** The velocity node is located exactly on the physical boundary. In this case, **direct substitution** is applied: the matrix row is set to 0 with a 1 on the main diagonal, and the prescribed boundary value is placed in the right-hand side (RHS).

4.1.2 Tangential Components (e.g., v or w in the X-sweep)

When solving for velocity components perpendicular to the sweep direction:

- **Left Boundary (Origin):** These nodes coincide with the physical boundary. Boundary conditions are imposed via **direct substitution** into the linear system.
- **Right Boundary:** The nodes are staggered relative to the wall. A **ghost node** u_{N+1} is introduced and eliminated from the equation for the last interior node N . Given the interior equation and the linear interpolation for the boundary value:

$$au_{N-1,j,k} + bu_{N,j,k} + cu_{N+1,j,k} = f \quad (9)$$

$$\frac{u_{N-1,j,k} + u_{N+1,j,k}}{2} = u_{ex,j,k} \quad (10)$$

The modified equation for node N becomes:

$$au_{N-1,j,k} + (b - c)u_{N,j,k} = f - 2cu_{ex,j,k} \quad (11)$$

4.2 Pressure Boundary Conditions

Pressure equations are solved through intermediate steps ψ , φ , and $\phi^{n+1/2}$ with homogeneous Neumann conditions. Due to the staggered grid layout, the distance between the wall and the nodes differs at the two ends, leading to different ghost node spacings.

4.2.1 Left Boundary (Origin)

The physical boundary is located exactly between the ghost node p_{-1} and the first interior node p_1 . A centered difference with a spacing of $2\Delta x$ is used to approximate the derivative:

$$\frac{p_{1,j,k} - p_{-1,j,k}}{2\Delta x} = \frac{\partial p}{\partial x} \Big|_{ex,j,k} \quad (12)$$

Substituting p_{-1} into the tridiagonal system $ap_{-1,j,k} + bp_{0,j,k} + cp_{1,j,k} = f$ results in the modified equation for node 0:

$$bp_{0,j,k} + (c + a)p_{1,j,k} = f + 2\Delta xa \frac{\partial p}{\partial x} \Big|_{ex,j,k} \quad (13)$$

4.2.2 Right Boundary

At the opposite end, the grid positioning requires the ghost node p_{N+1} to be eliminated using a difference with a spacing of Δx :

$$\frac{p_{N+1,j,k} - p_{N,j,k}}{\Delta x} = \frac{\partial p}{\partial x} \Big|_{ex,j,k} \quad (14)$$

This leads to the modified equation for the final node N :

$$ap_{N-1,j,k} + (b + c)p_{N,j,k} = f - \Delta xc \frac{\partial p}{\partial x} \Big|_{ex,j,k} \quad (15)$$

5 Serial Implementation

The solver is designed to simulate 3D fluid flow in porous media using the Navier-Stokes-Brinkman equations on a staggered grid. The serial implementation relies on a fractional step method (projection method) where the core computational effort is distributed across directional sweeps (X, Y, and Z). This approach simplifies complex 3D differential operators into sequences of independent 1D tridiagonal systems.

5.1 Core Data Structures

The solver relies on two primary data structures for discretized fields:

- **ScalarVariable**: Stores a 3D scalar field (e.g., pressure or velocity components) in a linearized array and provides indexed access and differential operators.
- **VectorVariable**: Groups three **ScalarVariable** objects to represent vector fields (u, v, w) and supports operations such as divergence and Laplacian evaluation.

5.2 Class Hierarchy

Solver responsibilities are divided to separate physical modeling from linear solves:

- **NavierStokesBrinkmann**: Manages time stepping, assembles momentum RHS terms, and performs the pressure projection.
- **Solver**: Abstract base class defining the interface for directional sweeps and storing grid geometry.
- **VelocitySolver** / **PressureSolver**: Implement momentum and pressure solves with appropriate coefficients and boundary conditions.
- **DimensionsHandler**: Provides index mapping between the 3D grid and 1D sweep lines.

5.3 Serial Algorithm Workflow

The serial execution follows a structured sequence of directional sweeps to update the fluid state from t_0 to T_{final} :

Algorithm 1 Fractional-Step Scheme with Directional Splitting (key stages)

Require: $\mathbf{u}^n, p^n, \Delta t$ (and coefficients β, γ)**Ensure:** $\mathbf{u}^{n+1}, p^{n+1}$ 1: **for** $t = 0$ to T_{final} **do** **Momentum predictor (velocity step)**2: $\boldsymbol{\xi}^{n+1} \leftarrow \mathbf{u}^n + \frac{\Delta t}{\beta} \hat{\mathbf{g}}^{n+1/2}$ $\triangleright$ explicit part3: Solve (X-sweep): $(I - \gamma \partial_{xx})(\boldsymbol{\eta}^{n+1} - \mathbf{u}^n) = \boldsymbol{\xi}^{n+1} - \mathbf{u}^n$ 4: Solve (Y-sweep): $(I - \gamma \partial_{yy})(\boldsymbol{\zeta}^{n+1} - \boldsymbol{\xi}^n) = \boldsymbol{\eta}^{n+1} - \boldsymbol{\xi}^n$ 5: Solve (Z-sweep): $(I - \gamma \partial_{zz})(\mathbf{u}^* - \mathbf{u}^n) = \boldsymbol{\zeta}^{n+1} - \mathbf{u}^n$ **Pressure projection (pressure step)**6: Solve (X-sweep): $(I - \partial_{xx})\psi = -\frac{1}{\Delta t} \nabla \cdot \mathbf{u}^*$ 7: Solve (Y-sweep): $(I - \partial_{yy})\varphi = \psi$ 8: Solve (Z-sweep): $(I - \partial_{zz})\phi^{n+1/2} = \varphi$ 9: $p^{n+1} \leftarrow p^n + \phi^{n+1/2}$ $\triangleright$ pressure update10: **end for**

Algorithm 2 Direction Splitting

Require: $\mathbf{u}^n, p^n, \Delta t$ (and coefficients β, γ)**Ensure:** $N_1 \notin$ considered direction**Ensure:** $N_2 \notin$ considered direction

5.4 Convergence Analysis

The numerical convergence of the implemented solver has been verified through a manufactured solution approach. A specific analytical solution for the velocity and pressure fields is chosen, and the corresponding source terms are derived to ensure that this solution satisfies the Navier-Stokes-Brinkman equations. By running simulations with progressively refined grids and time steps, the numerical solution is compared against the analytical solution. The errors are computed in appropriate norms (e.g., L^2 norm) to quantify the accuracy of the solver. The results confirm that the solver achieves second-order accuracy in space, as shown in Figure 2, the plot is consistent with the theoretical properties of the discretization scheme employed. This validation provides confidence in the correctness and reliability of the numerical implementation for simulating fluid flow in porous media. Similar convergence tests have been performed for the time discretization (Figure 3) underlining the

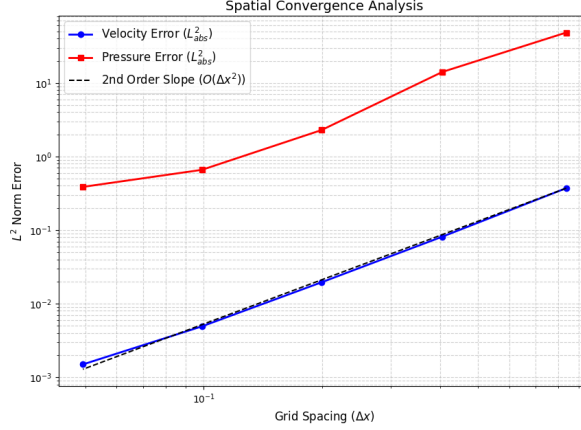


Figure 2: Spatial convergence analysis showing second-order accuracy.

Figure 3: Temporal convergence analysis showing second-order accuracy.

6 Parallelization

The numerical solution of the Navier–Stokes–Brinkman equations entails a high computational cost, making a sequential approach inefficient due to excessive computation times. While the Thomas algorithm (TDMA) is the standard method for solving tridiagonal systems in $\mathcal{O}(n)$ time on a single processor, its intrinsically sequential nature severely limits its scalability.

To overcome these limitations, a parallelization strategy based on the distributed-memory MPI (Message Passing Interface) paradigm has been implemented. The technological core of this implementation lies in the **Schur Complement Method**, a direct solver that enables efficient operation on distributed architectures while preserving numerical accuracy equivalent to that of the serial version.

Within the framework of Direction Splitting, a *line* is defined as the set of grid points aligned along the current computational direction, for example along the x -axis. The parallelization strategy is organized into two distinct and complementary levels:

- **Segment level (MPI):** The total length of each line is decomposed into local segments. Each MPI process is responsible for one segment and employs the Schur Complement Method to correctly handle the coupling with adjacent segments located on different processes. This approach allows the global

tridiagonal problem to be solved efficiently across distributed nodes.

- **Bundle level (OpenMP):** The three-dimensional domain is viewed as a collection of independent, parallel lines. Within each MPI process, OpenMP threads distribute the workload by simultaneously solving multiple lines. This shared-memory parallelism maximizes the utilization of available cores and significantly reduces the overall computation time of each time step.

6.1 Mathematical Foundations

Through domain decomposition, each process identifies its unknowns as either internal (u_i) or interface (u_s). The global system $Au = f$ can be rewritten in block-partitioned form as:

$$\begin{bmatrix} A_{ii} & A_{is} \\ A_{si} & A_{ss} \end{bmatrix} \begin{bmatrix} u_i \\ u_s \end{bmatrix} = \begin{bmatrix} f_i \\ f_s \end{bmatrix}. \quad (16)$$

Where:

- A_{ii} is a tridiagonal matrix containing the interactions among the internal points of a subdomain.
- A_{ss} represents the coefficients associated with the interface points located at the boundaries between processes.
- A_{is} and A_{si} describe the coupling between internal and interface unknowns.

By substituting the relation $u_i = A_{ii}^{-1}(f_i - A_{is}u_s)$ into the second equation, the reduced system (Schur complement) is obtained:

$$(A_{ss} - A_{si}A_{ii}^{-1}A_{is})u_s = f_s - A_{si}A_{ii}^{-1}f_i. \quad (17)$$

We define $S = (A_{ss} - A_{si}A_{ii}^{-1}A_{is})$ as the *Schur matrix* and $\hat{f}_s = f_s - A_{si}A_{ii}^{-1}f_i$ as the modified right-hand side. A key property of this formulation is that the Schur matrix S remains *tridiagonal*, allowing the reduced interface system to be solved efficiently using the Thomas algorithm.

6.2 Domain Decomposition

The solver architecture is orchestrated through a three-dimensional Cartesian domain decomposition, managed by the `MPITopology3D` class. This class creates a 3D Cartesian communicator using `MPI_Cart_create` and provides:

- A main communicator `cart3d_` for the full 3D domain with dimensions (P_z, P_y, P_x) .
- Three 1D sub-communicators `comm_` for line-wise solves:
 - `comm_x`: processes along the X direction (fixed p_y, p_z coordinates)
 - `comm_y`: processes along the Y direction (fixed p_x, p_z coordinates)
 - `comm_z`: processes along the Z direction (fixed p_x, p_y coordinates)

The domain is partitioned using a *balanced block distribution*. For a global dimension N distributed across P processes, each process p receives a contiguous block of points $[i_0(p), i_1(p))$ computed as follows:

$$q = \lfloor N/P \rfloor, \quad r = N \bmod P \quad (18)$$

$$i_0(p) = \begin{cases} p(q+1) & \text{if } p < r \\ r(q+1) + (p-r)q & \text{if } p \geq r \end{cases} \quad (19)$$

This ensures load balancing: the first r processes receive $(q+1)$ points each, while the remaining processes receive q points. The interface between two adjacent processes is shared by both, and each process allocates memory only for the points belonging to its own block, plus the interface points required for boundary computations.

6.3 Parallelized Algorithm Details

The implementation follows a five-phase workflow for each directional sweep, optimized to minimize MPI communication:

6.3.1 Phase 1–2: Local Computation (No MPI communication)

Each process independently performs:

1. **Internal solve:** Compute $y = A_{ii}^{-1} f_i$ using the Thomas algorithm on internal points.
2. **Interface RHS contribution:** Compute the local contribution to the modified right-hand side $\hat{f}_s = f_s - A_{si}y$.

Defining $nLines$ as the number of independent lines solved by each processor, we can exploit the OpenMP parallelization strategy as follows:

```
#pragma omp parallel for
for (i = 0; i < nLines) {
    compute_local_contrib();
}
```

6.3.2 Phase 3: MPI Collective Communication

In order to solve the Schur Complement, each local contribution must be shared among all the partitions in the same line. Since, within each domain decomposition, multiple lines are considered, the communication represents the bottleneck for this computation. Moreover, we adopt the following strategy :

A single `MPI_Allreduce` operation gathers and sums the interface RHS contributions from all processors:

```
comm_.allreduce();
```

This is the *only* MPI collective required per directional sweep, regardless of the number of lines being solved. The batched approach reduces the number of MPI calls from n_{lines} to exactly one, significantly improving parallel efficiency.

6.3.3 Phase 4–5: Local Finalization (No MPI communication)

Each process independently performs:

4. **Interface system solve:** Solve the reduced tridiagonal system $Su_s = \hat{f}_s$ locally using the Thomas algorithm. Since all processes have the complete interface RHS after the Allreduce, this redundant computation avoids an additional broadcast.
5. **Back-substitution:** Recover the internal unknowns $u_i = A_{ii}^{-1}(f_i - A_{is}u_s)$ and assemble the complete local solution.

These phases are also parallelized across lines using OpenMP.

6.4 Technical Deep Dive: C++ Implementation

The software architecture has been designed following object-oriented programming principles to ensure modularity and a clear separation of responsibilities. The main classes are:

6.4.1 SchurComplementSolver

This class represents the core of the parallel solver and manages the entire lifecycle of the computation:

- **preprocess(a, b, c)**: This function is crucial for computational efficiency. It computes the columns of the inverse matrix A_{ii}^{-1} required to form the Schur complement, assembles the local Schur contributions, and uses **MPI_Allgather** to build the global Schur matrix S . This preprocessing is performed once and reused for all subsequent solves with the same coefficients.
- **solve(rhs, solution)**: Implements the runtime phase for a single line, executing all five phases described above.
- **solve_batch(rhs_flat, nLines, sol_flat, use_omp)**: The optimized batched interface that processes multiple independent lines with a single MPI collective. This is the primary method used in production.

6.4.2 MPITopology3D

This class manages the 3D Cartesian topology and acts as an RAII wrapper for MPI communicators:

- Creates the 3D Cartesian communicator via **MPI_Cart_create**.
- Extracts 1D sub-communicators using **MPI_Cart_sub** for each spatial direction.
- Provides methods for computing local index ranges: **local_i0()**, **local_i1()**, **local_j0()**, **local_j1()**, **local_k0()**, **local_k1()**.
- Implements the balanced block distribution through the static method **block_range(N, p, P)**.
- Handles neighbor identification for potential halo exchanges.

6.4.3 MPICommunicator

This class acts as the communication manager and provides:

- Support for point-to-point operations (**send_value**, **recv_value**, **isend_value**, **irecv_value**).

- Collective operations optimized for the Schur complement method:
 - `allgather_schur_diag()`: Gathers Schur complement matrix contributions during preprocessing.
 - `allreduce_interface_rhs()`: Sums interface RHS values for a single line.
 - `allreduce_interface_rhs_batched()`: Sums interface RHS values for multiple lines in a single collective.
- Fallback to serial execution when running with a single process.

6.4.4 VelocitySolver and PressureSolver

These classes integrate the Schur complement solver into the Navier–Stokes–Brinkman framework:

- `solve_x_mpi()`, `solve_y_mpi()`, `solve_z_mpi()`: Direction-specific solve methods that:
 1. Create a `SchurComplementSolver` for the appropriate 1D communicator.
 2. Build the RHS locally (only for local points, ensuring $\mathcal{O}(N_{\text{local}})$ cost).
 3. Call `solve_batch()` with OpenMP parallelization over independent lines.
- Handle different boundary conditions (Dirichlet for normal components, ghost elimination for tangential components).

6.5 Batched Solver Optimization

A key optimization in the implementation is the batched solving approach. Instead of solving each 1D line independently (which would require one MPI collective per line), the solver processes all lines belonging to a process simultaneously:

1. **Phase 1–2:** All lines are processed in parallel using OpenMP, accumulating interface contributions into a flat buffer of size $n_{\text{lines}} \times n_{\text{interfaces}}$.
2. **Phase 3:** A single `MPI_Allreduce` operation handles all lines at once.
3. **Phase 4–5:** All lines are finalized in parallel using OpenMP.

This reduces the number of MPI collectives from $\mathcal{O}(N_y \times N_z)$ per sweep to exactly one, dramatically improving strong scaling efficiency. The implementation uses thread-local workspaces (`ThreadWorkspace`) to avoid synchronization overhead during the OpenMP parallel regions.

6.6 Parallelization Results

7 Conclusioni

Testo finale o commenti.